

Instituto Politécnico Nacional

Escuela Superior de Cómputo



Análisis y Diseño de Algoritmos

Práctica:

Algoritmo de Kruskal.

Fecha: 01/10/25

Domínguez Jiménez Ana Andrea

2. Ejercicios de programación.

Implementación de la estructura de conjuntos disjuntos:

```
}
Grafo* makeset(int u){ //crea un conjunto nuevo
    Grafo *nuevo=(Grafo*)malloc(sizeof(Grafo));
    if(nuevo==NULL) return NULL;

    nuevo->n=u;
    nuevo->raiz=(int*)malloc(u*sizeof(int));
    nuevo->nivel=(int*)malloc(u*sizeof(int));
    if(nuevo->raiz==NULL || nuevo->nivel==NULL){
        free(nuevo->raiz);
        free(nuevo->nivel);
        free(nuevo);
        free(nuevo);
        return NULL;
    }
    for(int i; i<u;i++){
        nuevo->raiz[i]=i;
        nuevo->nivel[i]=0; //apenas inicia
    }
    return nuevo;
}
```

Primero se verifica si no se a creado un conjunto antes, si es que no crea uno asignándole memoria a *nuevo,* después libera memoria y como es el primer conjunto que se crea la raíz apuntara al primer conjunto.

```
}

int find(Grafo *en, int x){
    if(en->raiz[x] != x){ //x no es su propio padre (hay mas de un conjunto ahora)
        en->raiz[x]= find(en,en->raiz[x]); //busca hasta encontrar al padre de x
    }
    return en->raiz[x];//devuelve a la raiz encontrada
}
```

Esta función busca la **raíz del conjunto** al que pertenece **x**, sigue los enlaces de padres hasta encontrar un nodo que **sea su propio padre**, esa raíz representa el conjunto.

```
59  void G_union (Grafo *arbol, int u, int v){
60      int xu = find(arbol, u);
61      int xv = find(arbol, v);
62      if(xu == xv){
63          printf("%d y %d estan en el mismo conjunto", xu, xv);
64      }
65      if(arbol->nivel[xu] < arbol->nivel[xv]){ //si se cumple, entonces xu se coloca abajo de xv
66          arbol->raiz[xu]=xv;
67      }else if(arbol->nivel[xu] > arbol->nivel[xv]){//se hace lo mismo pero al revez
68          arbol->raiz[xv]=xu;
69      }else{ //tienen el mismo tamaño(nivel)
70          arbol->raiz[xv]=xu;//xv esta abajo de xu ahora
71          arbol->nivel[xu]++; //se incrementa el nivel de la raiz
72      }
73  }
```

Luego la unión que primero llama a ***Find(x)*** y ***Find(y)***, si las raíces son diferentes entonces une los conjuntos.

3.)

```
}
void kruskal(Grafo *g, int u[], int v[], int w[], int a){
    MergeSort(w,a); //ordenar pesos
    printf("\nAristas ordenadas por peso:\n");
    for(int i=0;i < a;i++){
        printf("(%d, %d) peso %d\n",u[i], v[i], w[i]);
    }

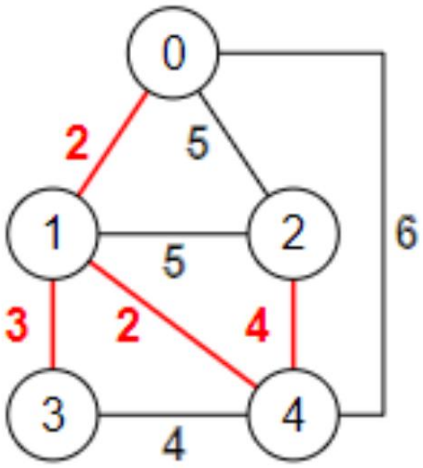
    int Xu[a], Xv[a];
    int cont=0;

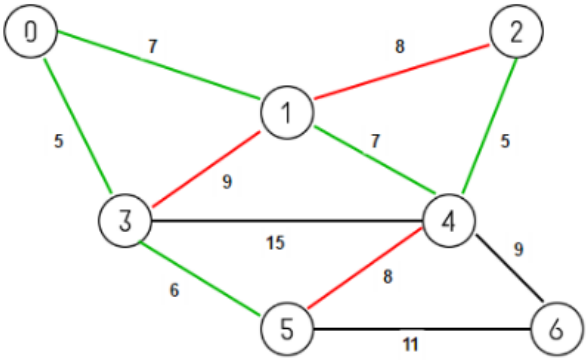

    for(int i=0; i<a;i++){
        int xu=find(g,u[i]);
        int xv=find(g, v[i]);
        printf("Arista (%d,%d), peso %d\n", u[i], v[i],w[i]);

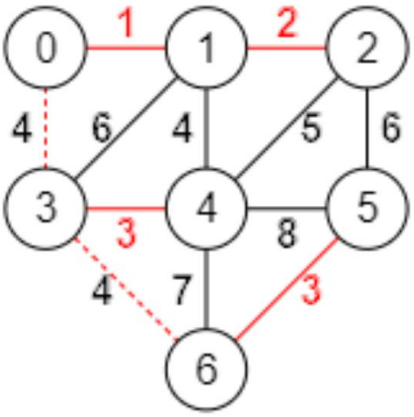
        if(xu!= xv){
            printf("No forma ciclo");
            G_union(g,xu,xv);
            cont++;
        }else{
            printf("Se forma un ciclo");
        }
    }
}
```

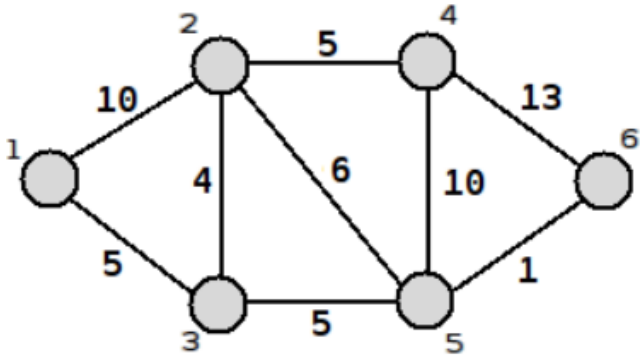
Kruskal se encarga de ordenar con ayuda de MergeSort los pesos de las aristas y después encontrar el camino con menos costos y después verificar que los grafos no formen un ciclo.

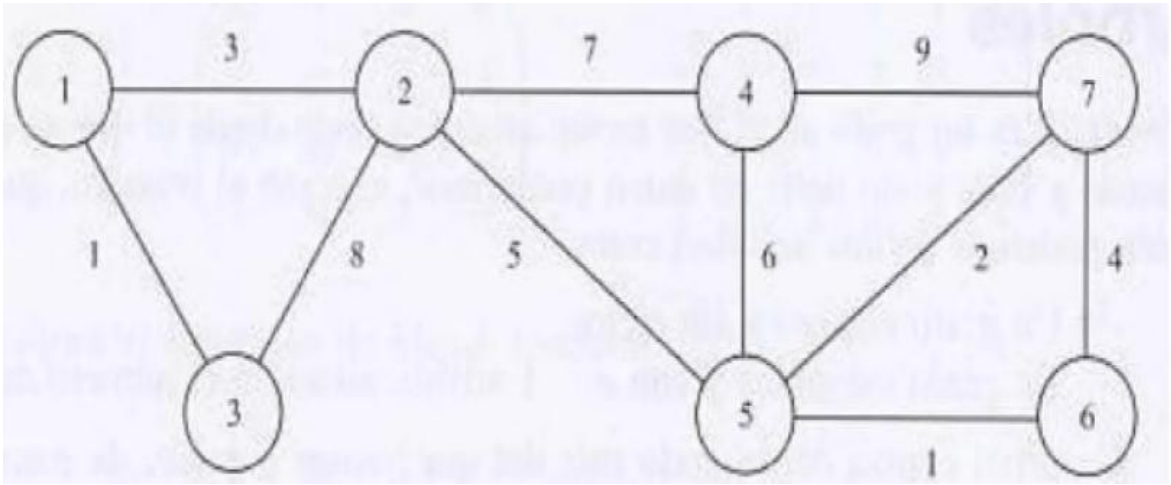
4.)


0
1
2
3
4
2
5
6
5
3
2
4
4


0
1
2
3
4
5
6
7
8
5
7
5
9
15
9
6
8
11


0
1
2
3
4
5
6
1
2
4
6
4
5
6
3
3
8
4
7
3


1
2
3
4
5
6
10
5
13
4
6
10
5
5
1


1
2
3
4
5
6
7
3
7
9
1
8
5
6
2
4
1